

Systems and Software Engineering I

DriftDetect

Melisa Lugji & Dilekcan Pamir

Goethe University Frankfurt am Main — Department of Computer Science and Mathematics

Winter Semester 2025/26

Table of Contents

Motivation and Problem

System Design

Detection and Simulation

Evaluation and Results

Visualization

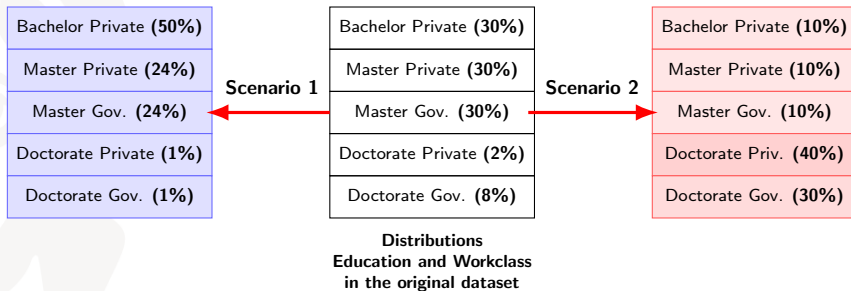
Discussion and Conclusion

Problem Context

- ▶ ML models are trained on historical data
- ▶ In production, data distributions often change over time
- ▶ This creates a mismatch between training and production data → **data drift**

Example of Data Drift

The model is supposed to predict whether a person's annual income exceeds \$50k.



- Impact: less reliable predictions and suboptimal decision-making

System-Level Problem

Detecting drift alone is not enough. A monitoring setup should also answer:

- ▶ When is drift detected?
- ▶ How reliable is detection?
- ▶ How many false alarms occur?

Basic Idea

- ▶ DriftDetect treats drift handling as a complete workflow
- ▶ It combines monitoring, simulation, evaluation, and visualization
- ▶ Goal: a modular and reproducible drift monitoring system

Functional Requirements

- ▶ Build a **baseline profile** from data
- ▶ Detect **numeric, categorical, and schema drift** on new data windows
- ▶ Simulate **scenario-driven drift** from YAML scenario definitions
- ▶ Compute **coverage metrics**: FPR, detection power, TTD, and missed events
- ▶ Produce **JSON artifacts**, an HTML report, and dashboard views

Non-Functional Requirements

- ▶ **Reproducibility:** same inputs and seed reproduce simulation outputs
- ▶ **Modularity:** simulation, detection, evaluation, and visualization are separated
- ▶ **CLI-based workflow:** commands support baseline, detection, simulation, coverage, reporting
- ▶ **Explainability:** feature-level metrics and summaries
- ▶ **Portability:** file-based artifacts usable across environments

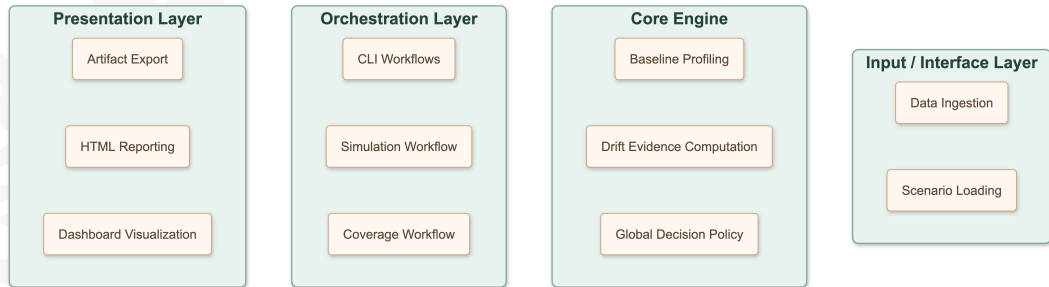
Design Principle

DriftDetect is an **artifact-driven drift monitoring workflow** for **tabular data**.

- ▶ Intermediate results are stored as JSON artifacts
- ▶ Each stage consumes the previous stage's outputs
- ▶ This keeps the pipeline transparent and reproducible

Architecture Overview

The system is organized into four logical layers:

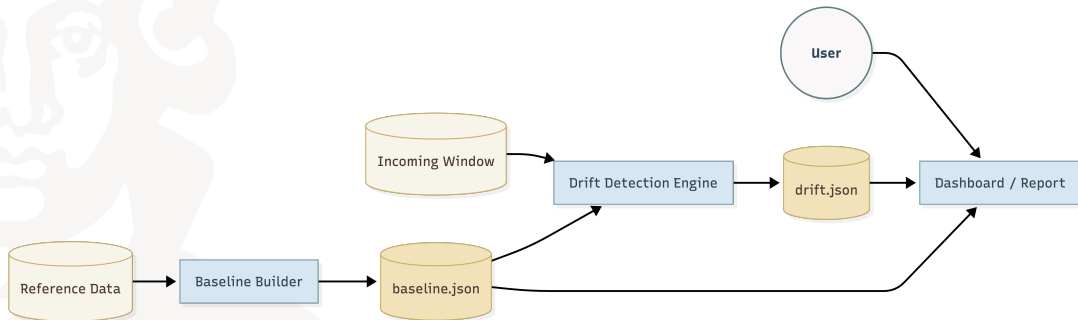


Operating Mode

1. Production mode (Monitoring)

- ▶ baseline → detect drift

V-model: build side

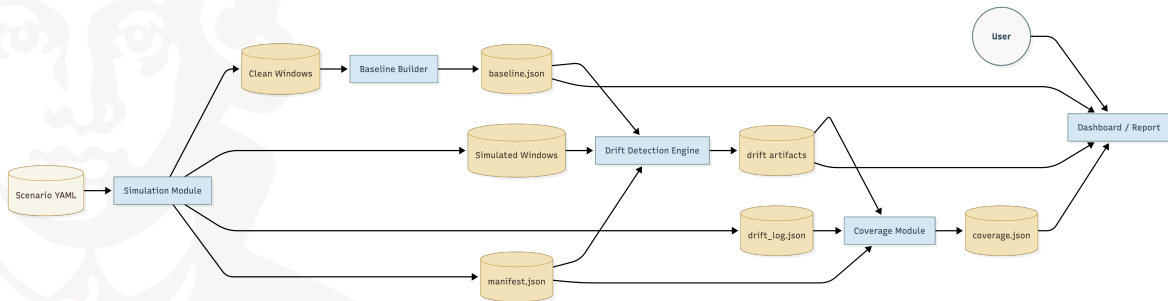


Operating Mode

2. Engineering mode (Validation / Evaluation)

► simulate → baseline → detect drift → evaluate

V-model: verify side



Baseline Profile

- ▶ Built from clean reference data
- ▶ Stores the normal state of the dataset
- ▶ Includes schema, numeric summaries, and categorical frequencies
- ▶ Output artifact: `baseline.json`

Used as the **reference** for all later drift comparisons.

Drift Detection Approach

Detection is performed at two levels:

1. **Schema level**

- ▶ added, removed, or type-changed features

2. **Feature level**

- ▶ numeric drift: distribution, mean/variance, missingness
- ▶ categorical drift: distribution, new categories, missingness

Detection Workflow



Statistical Tests

Complementary evidence per feature type.

Numeric Features

► Kolmogorov-Smirnov

Maximum gap between cumulative distribution functions; captures distribution shape changes.

► Wasserstein Distance

Measures how far probability mass must move; sensitive to shifts across the full range.

Numeric severity = $\max(s_{KS}, s_{WD}, s_{mean}, s_{std}, s_{miss})$

Categorical severity = $\max(s_{JS}, s_p, s_{miss})$

Categorical Features

► Chi-Square

Tests whether observed category counts match expected baseline frequencies.

► Jensen-Shannon Divergence

Measures distance between category distributions and remains stable with new categories.

Global Decision Logic

Parameters

- ▶ S_{feat} : feature severity threshold
- ▶ S_{global} : global severity threshold
- ▶ K : minimum drifted features

Step 1: Feature-Level Decision

```
if feature_severity  $\geq S_{\text{feat}}$ :
    mark feature as drifted
```

Step 2: Global Decision

```
if schema_drift present:
    flag window
else if max_severity  $\geq S_{\text{global}}$ :
    flag window
else if drifted_features  $\geq K$ :
    flag window
else:
    no drift
```

Simulation in Engineering Mode

- ▶ Input: controlled drift scenario .yaml
- ▶ Supports **abrupt** and **gradual** drift modes
 - ▶ windows/window_000.csv ... window_011.csv → **baseline building** and **detection**
 - ▶ manifest.json → **detection** and **coverage**
 - ▶ drift_log.json → **coverage**
- ▶ Two dataset themes: **income** (demographic) and **logistics** (order data)

Income Simulation — Results

Synthetic demographic dataset (age, income, hours_per_week, occupation, marital_status).

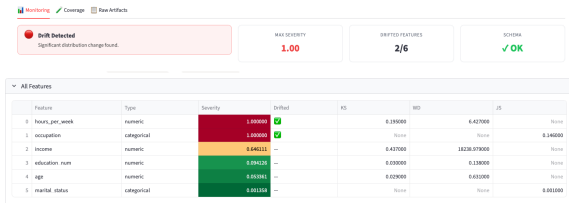
12 windows · 600 rows each · 4 clean baseline windows.

Event	Drift Type	Power	TTD	Detected by
Income gradual shift	numeric_shift	1.0	2 windows	KS + Wasserstein
New occupation category	new_category	1.0	0 windows	JSD = 1.0
Hours variance drift	variance_shift	1.0	0 windows	Wasserstein

FPR = 0.0 Overall Power = 1.0

Gradual drift (income shift ramped over 4 windows) took 2 windows longer to detect than abrupt drift — detection latency depends on drift speed.

Detection in Action — Income Experiment

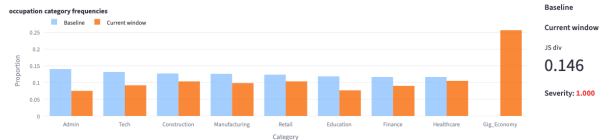


- ▶ **hours_per_week**: severity 1.0 — variance doubled
- ▶ **occupation**: severity 1.0 — Gig_Economy appeared, JSD fires immediately
- ▶ **income**: severity 0.646 — gradual shift, below threshold
- ▶ **age, education, marital**: near zero — no drift injected

Color gradient: red → yellow → green maps directly to severity score.

New Category Detection — Occupation

occupation — Distribution

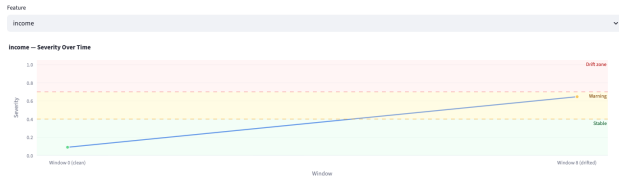


- ▶ Blue bars = baseline frequencies
- ▶ Orange bars = current window
- ▶ Gig_Economy has **no blue bar** — never in training data
- ▶ $JSD = 0.146$,
Severity = 1.000

JSD handles new categories naturally — zero baseline probability does not cause numerical errors.

Gradual Drift — Income Feature Over Time

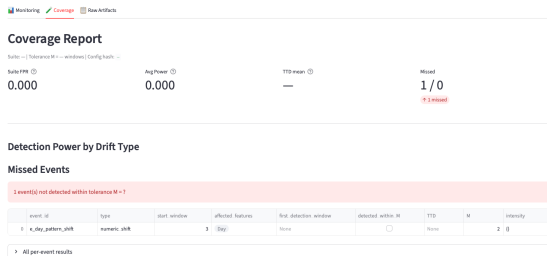
Severity Over Time



- ▶ Window 0: severity 0.09 (clean)
- ▶ Window 8: severity 0.646 (full drift)
- ▶ Climbs gradually through warning zone
- ▶ Does not cross 0.7 threshold — $TTD = 2$

Gradual drift is harder to catch early. Coverage metrics quantify exactly how much harder.

Key Finding — Monitoring Blind Spot



Day pattern shift: CUST_A moves Tue → Fri

- ▶ FPR = 0.000 — no false alarms
- ▶ Power = **0.000** — never detected
- ▶ TTD = ∞ — never fired
- ▶ 1 missed event

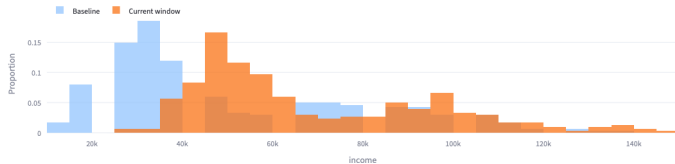
Why? CUST_B orders on all 7 days, diluting the Day signal. One customer's shift becomes invisible when pooled.

Fix: Monitor per-customer sub-windows. Coverage identified the blind spot and points to the solution.

Dashboard — Distribution Comparison

income — Distribution

income distribution



Baseline

Mean

51952.17

Std

28228.88

Median

40209.82

Current window

KS stat

0.437

WD

18238.979

Severity: 0.646

Blue = baseline Orange = drifted window (income shifted right by \$18,238)

Dashboard — Features

- ▶ **Artifact-only:** reads JSON files and never recomputes drift
- ▶ **Experiment selector:** switch between income, logistics, and demo experiments
- ▶ **Severity Over Time:** time-series chart with green/yellow/red threshold bands
- ▶ **Distribution Comparison:** baseline vs. current window
- ▶ **Feature Table:** features ranked by severity with relevant metrics and color gradient
- ▶ **Coverage Report:** FPR, Power, TTD, and missed events when ground truth is available

Limitations and Future Work

Current Limitations

- ▶ **Univariate tests only** — feature relationships are not fully captured
- ▶ **Fixed baseline** — gradual drift is detected more slowly; a rolling baseline could improve sensitivity
- ▶ **Threshold tuning** — optimal thresholds depend on the application domain

Future Work

- ▶ Multivariate drift detection (e.g. MMD, correlation monitoring)
- ▶ Adaptive threshold selection based on historical evaluation results
- ▶ Real-time streaming mode instead of only batch/window-based processing
- ▶ Per-segment monitoring (e.g. per customer) to avoid signal dilution

Conclusion

- ▶ DriftDetect is a **complete drift monitoring pipeline** for tabular data
- ▶ It combines detection, simulation, evaluation, and visualization in one workflow
- ▶ Feature-level evidence is unified through interpretable severity scores
- ▶ Simulation with ground truth makes detector quality measurable through FPR, Power, and TTD
- ▶ **Concrete finding:** the logistics day-pattern shift was completely missed (**Power = 0.0**). Simulation and coverage analysis exposed this blind spot, which would likely remain invisible in production-only monitoring.

Thank you for your attention!

